AI-ML Engineering Assessment: Construction Data Automation

Overview

This assignment simulates the real-world operational challenges faced daily in the construction technology sector. The objective is to evaluate your ability to architect resilient systems, handle messy physical-world data, and optimize for both performance and cloud API costs.

Guidelines & Rules of Engagement:

- **Time Allocation:** We expect this to take roughly 4–6 hours. Please prioritize architectural soundness and handling edge cases over perfecting minor details.
- **API Costs & Third-Party Tools:** You are free to use any open-source models, cloud APIs (OpenAI, Anthropic, AWS, GCP, etc.), or frameworks you prefer. **If a solution requires paid APIs and you do not have credits, you do not need to spend your own money.** You may build the pipeline and leave a mock placeholder function for the API call, provided you document exactly how the API would be configured (e.g., system prompts, structured output schemas, parameters).
- **Submission:** Submit your solution via a GitHub repository link or a zipped file. It must include your source code, the requested outputs, and a README.md explaining your architectural choices.



Task 1: Field Data Digitization Pipeline (Code Execution)

The Problem

Hundreds of handwritten safety observation forms are collected directly from construction sites every day. These forms are often messy, folded, and filled with construction slang.

You have been provided with:

1. **A sample input PDF** containing scanned, handwritten forms (including edge cases like corrupted or explicitly failed test forms).
2. **A master template image (template.jpg)** showing the clean, blank structure of the form for reference.

The Challenge

Handwritten field forms are notoriously difficult to digitize reliably at scale. Physical data is unpredictable you will encounter heavily skewed pages, dirty or muddy forms, and crossed out text.

Your goal is to architect a robust pipeline that takes these raw, unstructured inputs, extracts the relevant checkboxes and text fields, and maps the data to a clean, structured JSON format. You are free to use any combination of computer vision techniques, OCR engines, or Vision-Language Models to accomplish this.

Deliverables for Task 1

1. **Extraction Script:** Provide the Python code that processes the sample PDF.
2. **Output JSON:** The script must output a structured JSON file containing the extracted data (e.g., Date, Observer, Checkbox States, Safety Comments).
3. **Resilience Strategy:** Briefly explain in your README how your pipeline handles "garbage" inputs (e.g., a completely blank, muddy, or corrupted page) to prevent downstream pipeline crashes or hallucinated data.

1. Task 2: P6 Schedule Batch Classification (Architecture)

The Problem

We ingest Primavera P6 construction schedules containing tens of thousands of activity rows. Often, the client has assigned the wrong discipline or sub-discipline to these activities in their raw data.

You have been provided with a CSV containing Activity ID and Activity Name. Your system must analyze the Activity Name and correctly assign it to the strict, predefined taxonomy of **Discipline** and **Sub-Discipline** listed below.

The Taxonomy Rubric

Your system is strictly constrained to this taxonomy. The AI cannot invent or hallucinate new categories.

- **Architectural:** Acoustical Ceiling Tile, Exterior Finishes, Hot Aisle Containment, Interior Finishes, Miscellaneous
- **Building Automation System:** Pull, Test and Terminate
- **Civil:** Exterior Equipment Pad, Foundation Works, Miscellaneous, Slab on Grade, Underground Dry Utilities, Underground Wet Utilities
- **Electrical:** Cable Pulling, Cable Termination, Cable Trays Drops, Cable Trays Main, Miscellaneous, Rough-In
- **Fire Alarm:** Pull, Test and Terminate
- **Fire Protection:** Installation
- **Landscape:** Miscellaneous
- **Mechanical:** Exterior Installation, Rough-In
- **Plumbing:** Rough-In
- **Security:** Pull, Test and Terminate
- **Structural:** Envelope, Exterior Equipment Steel, Miscellaneous
- **Telecommunications:** Basket Tray Main, Digital Antenna System / Radio Over IP, Miscellaneous, Production Fiber

The Challenge

Running 50,000+ individual rows sequentially through an LLM is slow, financially expensive, and highly prone to hallucinations (inventing sub-disciplines outside the rubric).

Deliverables for Task 2

For this task, **you are not required to build a working application**, though you may include a proof-of-concept script if you wish. The primary deliverable is an **Architecture Document** (to be included in your README) addressing the following:

1. **Pipeline Architecture:** How would you design a scalable batch processing pipeline to classify tens of thousands of rows quickly?
2. **Constraint Enforcement:** How do you guarantee the AI strictly selects from the provided rubric and does not hallucinate new disciplines?
3. **Cost & Accuracy Optimization:** Instead of passing every single isolated text string to an expensive reasoning model, how would you architect the system to reduce API costs and latency while maintaining high accuracy?